

RS001 Week 8 — English Worksheet

Topic: Validation Across the Whole Game · **Course:** Text Adventure (Python) · **Time:** about 45 minutes

This week you think about wrapping the "clean → validate → retry → default" pattern into **one reusable function** called `safe_choice()`. You read it, trace it, fix broken versions of it, and then explain — in your own words — the code you wrote in today's live lesson.

 Words to know: **function**, **return**, **reuse**, **default**, **valid**

```
def safe_choice(prompt, valid_list, default):
    answer = game.ask(prompt).strip().lower()
    if answer not in valid_list:
        answer = game.ask("Try again: " + prompt).strip().lower()
    if answer not in valid_list:
        answer = default
    return answer
```

1 · Predict

Read each piece of code. In your head, work out what happens, then write your answer.

```
def greet(name):
    return f"Hi {name}!"

game.say(greet("Mina"))
```

A function with `return`. What is printed?


```
def safe_choice(prompt, valid_list, default):
    answer = game.ask(prompt).strip().lower()
    if answer not in valid_list:
        answer = default
    return answer

result = safe_choice("way?", ["left", "right"], "left")
game.say(result)
```

The player types `"up"`. What does `result` become?


```
direction = safe_choice("way?", ["left", "right"], "left")
action = safe_choice("do?", ["fight", "run"], "run")
game.say(f"{direction}, {action}")
```

The same function is reused twice with different lists. Why is reusing it better than copying the code?

2 · Spot the Bug 🐛

Each block was meant to do something, but it is broken. Read what it is **supposed** to do, fix it on paper, then explain why the original was wrong.

Bug A — `safe_choice` should hand the answer back to whoever called it. Right now it cleans the answer but forgets to `return` it, so the caller gets nothing.

```
def safe_choice(prompt, valid_list, default):
    answer = game.ask(prompt).strip().lower()
    if answer not in valid_list:
        answer = default

result = safe_choice("way?", ["left", "right"], "left")
game.say(result)
```

Hint: a function needs `return` to give a value back.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug B — The default must itself be a **valid** answer, or the game ends up holding something not in the list. Here the default `"up"` is not in the valid list.

```
def safe_choice(prompt, valid_list, default):
    answer = game.ask(prompt).strip().lower()
    if answer not in valid_list:
        answer = default
    return answer

move = safe_choice("way?", ["left", "right"], "up")
game.say(f"You go {move}.")
```

Hint: pick a default that lives inside the valid list.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug C — The valid list should be lowercase so the cleaned answer can match it. Here `"Fight"` has a capital, so a cleaned `"fight"` is never accepted.

```
action = safe_choice("do what?", ["Fight", "run"], "run")
game.say(f"You chose {action}.")
```

Hint: the input gets `.lower()`-ed, so the list items must be lowercase too.

Write the fixed code:

Why was it wrong? Why does your fix work?

3 · Explain the Code

Here is the full `safe_choice` function from this week. Read it slowly, then answer the questions below.

```
def safe_choice(prompt, valid_list, default):  
    answer = game.ask(prompt).strip().lower()  
    if answer not in valid_list:  
        answer = game.ask("Try again: " + prompt).strip().lower()  
    if answer not in valid_list:  
        answer = default  
    return answer
```

What do `.strip()` and `.lower()` do to the player's answer, and why are they used together here?

The function asks a second time before giving up. Which line does the asking-again, and when does it happen?

If the player still gives a bad answer after the second try, what value does `answer` end up holding?

Why does the last line say `return answer` instead of `game.say(answer)` ?

The three inputs are `prompt` , `valid_list` , and `default` . In one sentence each, what job does each one do?

4 · Explain Your Lesson Code 🎥

In today's live lesson you wrote your own `safe_choice()` and used it in your game. Now explain **your** code. Take a short video on your phone (or a parent's phone) and teach it like the viewer has never coded. You may show it running. Try to use these words: **function**, **return**, **reuse**, **default**, **valid**.

1. Read your `safe_choice` function out loud and say what each part does.
2. Point to a place where you reused it, and explain why reusing beats copying.
3. Explain what your `default` is and why it is a valid answer.
4. Explain why the function ends with `return` .

Write what you will say in your video. Plan it here before you record.

Submit 

Send this worksheet + a video explaining your lesson code to teacher on KakaoTalk.